

SMART STUDY PLANNER

An Academic Productivity System

Source Code Submission

(First and Last Portions of Source Code)

Prepared for Copyright Submission

Authors:

Purab Vats
Taranpreet Singh
Tanish Singla
Shalesh Thakur
Shikha Tuteja

File: frontend/_App.jsx

```
import { Navigate, Route, Routes } from 'react-router-dom';
import LandingPage from './pages/Landing/LandingPage';
import LoginPage from './pages/Login/LoginPage';
import RegisterPage from './pages/Register/RegisterPage';
import DashboardPage from './pages/Dashboard/DashboardPage';
import SubjectsPage from './pages/Subjects/SubjectsPage';
import TasksPage from './pages/Tasks/TasksPage';
import MentalLoadPage from './pages/MentalLoad/MentalLoadPage';
import SchedulePage from './pages/Schedule/SchedulePage';
import AnalyticsPage from './pages/Analytics/AnalyticsPage';
import ProtectedRoute from './components/ProtectedRoute';

function App() {
  return (
    <Routes>
      <Route path="/" element={<LandingPage />} />
      <Route path="/login" element={<LoginPage />} />
      <Route path="/register" element={<RegisterPage />} />
      <Route
        path="/dashboard"
        element={
          <ProtectedRoute>
            <DashboardPage />
          </ProtectedRoute>
        }
      />
      <Route
        path="/subjects"
        element={
          <ProtectedRoute>
            <SubjectsPage />
          </ProtectedRoute>
        }
      />
      <Route
        path="/tasks"
        element={
          <ProtectedRoute>
            <TasksPage />
          </ProtectedRoute>
        }
      />
      <Route
        path="/mental-load"
        element={
```

```

        <ProtectedRoute>
          <MentalLoadPage />
        </ProtectedRoute>
      }
    />
    <Route
      path="/schedule"
      element={
        <ProtectedRoute>
          <SchedulePage />
        </ProtectedRoute>
      }
    />
    <Route
      path="/analytics"
      element={
        <ProtectedRoute>
          <AnalyticsPage />
        </ProtectedRoute>
      }
    />
    <Route path="*" element={<Navigate to="/" replace />} />
  </Routes>
);
}

```

```
export default App;
```

File: frontend/_src/ pages/ Landing/ LandingPage.jsx

```

// LandingThemeToggle component - renders theme toggle button (Light/Dark mode)
// Takes onRegister callback to track button elements for animations
function LandingThemeToggle({ onRegister }) {
  const { theme, toggleTheme } = useTheme();

  return (
    <button
      type="button"
      className="ssp-landing__theme-toggle"
      onClick={toggleTheme}
      ref={onRegister}
      aria-label={`Switch to ${theme === 'dark' ? 'light' : 'dark'} mode`}
    >
      <span className="ssp-landing__theme-thumb">
        <span className="ssp-landing__theme-icon">{theme === 'dark' ? '●' : '○'}</span>
      </span>
      <span className="ssp-landing__theme-label">{theme === 'dark' ? 'Dark' :

```

```

'Light'</span>
    </button>
  );
}

```

// Main LandingPage component - landing page with animations, features showcase and CTAs

```

export default function LandingPage() {
  const { theme } = useTheme();
  // Track scroll state to apply sticky navigation styling
  const [navScrolled, setNavScrolled] = useState(false);
  // Refs for GSAP animation context
  const rootRef = useRef(null);
  const heroArtRef = useRef(null);
  const deviceRef = useRef(null);
  const particleRefs = useRef([]);
  const floatingRefs = useRef([]);
  const cardRefs = useRef([]);
  const buttonRefs = useRef([]);

  // Register particle elements for animation reference
  const registerParticle = (element) => {
    if (element && !particleRefs.current.includes(element))
particleRefs.current.push(element);
  };

  // Register floating elements for animation reference
  const registerFloating = (element) => {
    if (element && !floatingRefs.current.includes(element))
floatingRefs.current.push(element);
  };

  // Register card elements for animation reference
  const registerCard = (element) => {
    if (element && !cardRefs.current.includes(element)) cardRefs.current.push(element);
  };

  // Register button elements for animation reference
  const registerButton = (element) => {
    if (element && !buttonRefs.current.includes(element))
buttonRefs.current.push(element);
  };

  // useEffect - Handle scroll event to toggle navbar styling when scrolled
  // Updates navScrolled state based on scroll position
  useEffect(() => {
    const onScroll = () => setNavScrolled(window.scrollY > 18);
    onScroll();
  });
}

```

```
window.addEventListener('scroll', onScroll, { passive: true });  
return () => window.removeEventListener('scroll', onScroll);  
}, []);
```

```
// Add hover and click animations to buttons  
// Scale effect on hover and press animation on click  
buttonRefs.current.forEach((button) => {  
  const hoverIn = () =>  
    gsap.to(button, {  
      scale: 1.05,  
      duration: 0.2,  
      ease: 'power2.out'  
    });  
  
  const hoverOut = () =>  
    gsap.to(button, {  
      scale: 1,  
      duration: 0.22,  
      ease: 'power2.out'  
    });  
  
  const press = () =>  
    gsap.to(button, {  
      scale: 0.98,  
      duration: 0.08,  
      ease: 'power2.out',  
      yoyo: true,  
      repeat: 1  
    });  
  
  button.addEventListener('mouseenter', hoverIn);  
  button.addEventListener('mouseleave', hoverOut);  
  button.addEventListener('click', press);  
});  
  
// Add 3D perspective effect to device on mouse movement  
// Device rotates based on mouse position within hero section  
// Creates depth and interactivity effect  
if (heroArtRef.current && deviceRef.current) {  
  const handleMove = (event) => {  
    const bounds = heroArtRef.current.getBoundingClientRect();  
    const x = (event.clientX - bounds.left) / bounds.width - 0.5;  
    const y = (event.clientY - bounds.top) / bounds.height - 0.5;  
  
    gsap.to(deviceRef.current, {  
      rotateY: x * 8,  

```

```

        rotateX: -y * 8,
        duration: 0.35,
        ease: 'power2.out'
    });
};

const reset = () => {
    gsap.to(deviceRef.current, {
        rotateY: 0,
        rotateX: 0,
        duration: 0.45,
        ease: 'power2.out'
    });
};

heroArtRef.current.addEventListener('mousemove', handleMove);
heroArtRef.current.addEventListener('mouseleave', reset);
}
}, rootRef);

return () => ctx.revert();
}, [theme]);

// Smooth scroll to section function - handles navigation to page sections
// Accounts for navbar height offset for proper positioning
const scrollToSection = (event, id) => {
    event.preventDefault();
    const section = document.getElementById(id);
    if (!section) return;

    const navOffset = 112;
    const top = section.getBoundingClientRect().top + window.scrollY - navOffset;
    window.scrollTo({ top, behavior: 'smooth' });
};

// JSX Structure - Main landing page container
return (
    <div className={`ssp-landing theme-${theme}`} ref={rootRef}>
        <div className="ssp-landing_bg">
            <span className="ssp-landing__particle particle-a" ref={registerParticle} />
            <span className="ssp-landing__particle particle-b" ref={registerParticle} />
            <span className="ssp-landing__particle particle-c" ref={registerParticle} />
            <span className="ssp-landing__particle particle-d" ref={registerParticle} />
            <span className="ssp-landing__grid" />
        </div>

        </* Navigation Header - Sticky navbar with theme toggle and auth links */>

```

```

<header className={`ssp-landing__nav ${navScrolled ? 'is-scrolled' : ''}`}>
  <Link to="/" className="ssp-landing__brand">
    <span className="ssp-landing__brand-mark">
      <PlannerLogoMark size={36} accent={theme === 'dark' ? 'green' : 'dark'} />
    </span>
    <span>
      <strong>Smart Study Planner</strong>
      <small>Academic productivity system</small>
    </span>
  </Link>

  <nav className="ssp-landing__links">
    <a href="#features" onClick={(event) => scrollToSection(event, 'features')}>Features</a>
    <a href="#how-it-works" onClick={(event) => scrollToSection(event, 'how-it-works')}>How it works</a>
    <a href="#benefits" onClick={(event) => scrollToSection(event, 'benefits')}>Benefits</a>
    <a href="#cta" onClick={(event) => scrollToSection(event, 'cta')}>Get started</a>
  </nav>

  <div className="ssp-landing__nav-actions">
    <LandingThemeToggle onRegister={registerButton} />
    <Link to="/login" className="ssp-landing__ghost-btn" ref={registerButton}>Log in</Link>
    <Link to="/register" className="ssp-landing__primary-btn" ref={registerButton}>Get Started</Link>
  </div>
</header>

<main className="ssp-landing__main">
  /* Hero Section - Main headline with call-to-action and animated device mockup
*/
  <section className="ssp-landing__hero">
    <div className="ssp-landing__hero-copy reveal-on-scroll">
      <span className="ssp-landing__eyebrow">Plan smarter. Study calmer.</span>
      <h1 className="ssp-landing__headline">
        <span className="headline-word">Your</span>{' '}
        <span className="headline-word headline-word-accent">Smart</span>{' '}
        <span className="headline-word">Study</span>{' '}
        <span className="headline-word">Partner</span>
      </h1>
      <p>
        Organize subjects, adapt study intensity to real workload, and keep progress visible
        without turning your week into a burnout experiment.
      </p>
      <div className="ssp-landing__hero-actions">

```

```

        <Link to="/register" className="ssp-landing__primary-btn"
ref={registerButton}>Get Started Free</Link>
        <a
            href="#how-it-works"
            className="ssp-landing__watch-btn"
            ref={registerButton}
            onClick={(event) => scrollToSection(event, 'how-it-works')}
        >
            <span className="ssp-landing__watch-icon">▶</span>
            Watch Demo
        </a>
    </div>
</div>

```

```

<div className="ssp-landing__hero-art reveal-on-scroll" ref={heroArtRef}>
    <div className="ssp-landing__hero-glow" />
    <div className="ssp-landing__orbit orbit-a" />
    <div className="ssp-landing__orbit orbit-b" />
    <div className="ssp-landing__plinth" />

    <div className="ssp-landing__device" ref={deviceRef}>
        <div className="ssp-landing__device-top">
            <span className="ssp-landing__device-notch" />
            <span className="ssp-landing__device-camera" />
        </div>
        <div className="ssp-landing__device-screen">
            <div className="ssp-landing__device-summary">
                <div>
                    <strong>Today's plan</strong>
                    <span>Adaptive workload balance</span>
                </div>
                <div className="ssp-landing__progress-ring">75%</div>
            </div>

            <div className="ssp-landing__task-card tone-blue">
                <b>Math Focus</b>
                <small>9:00 AM - 10:30 AM</small>
            </div>
            <div className="ssp-landing__task-card tone-violet">
                <b>Physics Review</b>
                <small>11:00 AM - 12:15 PM</small>
            </div>
            <div className="ssp-landing__task-card tone-green">
                <b>Recovery Break</b>
                <small>12:30 PM - 1:00 PM</small>
            </div>
            <div className="ssp-landing__task-card tone-indigo">

```



```

        <b>Chemistry Drill</b>
        <small>3:00 PM - 4:00 PM</small>
    </div>
</div>
</div>

<div className="ssp-landing__floating-book" ref={registerFloating}>
    <span className="ssp-landing__book-top" />
    <span className="ssp-landing__book-pages" />
</div>

<div className="ssp-landing__floating-clock" ref={registerFloating}>
    <span className="clock-face" />
    <span className="clock-hand short" />
    <span className="clock-hand long" />
</div>

<div className="ssp-landing__floating-cap" ref={registerFloating}>
    <span className="cap-top" />
    <span className="cap-base" />
    <span className="cap-thread" />
</div>

<div className="ssp-landing__glass-note note-left" ref={registerFloating}>
    <strong>Balanced study load</strong>
    <span>Heavy sessions automatically shifted away from high-fatigue
windows.</span>
</div>

<div className="ssp-landing__glass-note note-right" ref={registerFloating}>
    <strong>Overload signals</strong>
    <span>Spot missed tasks, pressure spikes, and recovery gaps before they
stack.</span>
</div>
</div>
</section>

    { /* Features Section - Displays key product features with icons and
descriptions */ }
    <section id="features" className="ssp-landing__section reveal-on-scroll">
        <div className="ssp-landing__section-head">
            <span className="ssp-landing__eyebrow">Features</span>
            <h2>Everything you need to run an academic system, not just a to-do
list.</h2>
        </div>
        <div className="ssp-landing__feature-grid">
            {featureCards.map((feature) => (
                <article key={feature.title} className="ssp-landing__feature-card"

```

```

ref={registerCard}>
    <div className="ssp-landing__feature-icon">
        <IconGlyph type={feature.icon} />
    </div>
    <h3>{feature.title}</h3>
    <p>{feature.copy}</p>
</article>
    )}}
</div>
</section>

    { /* How It Works Section - Step-by-step guide showing user journey through the
app */ }
    <section id="how-it-works" className="ssp-landing__section reveal-on-scroll">
        <div className="ssp-landing__section-head">
            <span className="ssp-landing__eyebrow">How it works</span>
            <h2>Plan. Study. Adapt. Improve.</h2>
        </div>
        <div className="ssp-landing__steps-line" aria-hidden="true" />
        <div className="ssp-landing__steps-grid">
            {steps.map((step) => (
                <article key={step.number} className="ssp-landing__step-card"
ref={registerCard}>
                    <span className="ssp-landing__step-badge">{step.number}</span>
                    <div className="ssp-landing__step-icon">
                        <IconGlyph type={step.icon} />
                    </div>
                    <h3>{step.title}</h3>
                    <p>{step.copy}</p>
                </article>
            )}}
        </div>
    </section>

    { /* Benefits Section - Highlights unique value propositions and main benefits
*/ }
    <section id="benefits" className="ssp-landing__section reveal-on-scroll">
        <div className="ssp-landing__benefit-panel" ref={registerCard}>
            <div className="ssp-landing__benefit-copy">
                <span className="ssp-landing__eyebrow">Benefits</span>
                <h2>A planning surface that respects focus, deadlines, and recovery
equally.</h2>
                <p>
                    Smart Study Planner combines scheduling, workload control, mental-load
awareness,
                    and actionable analytics into one clean workspace.
                </p>
            </div>

```

```

    <div className="ssp-landing__benefit-stats">
      <div>
        <strong>Adaptive schedule engine</strong>
        <span>Balances subject load across the week.</span>
      </div>
      <div>
        <strong>Fatigue-aware planning</strong>
        <span>Reduces heavy sessions on recovery-focused days.</span>
      </div>
      <div>
        <strong>Clean analytics layer</strong>
        <span>Makes productivity patterns easy to understand.</span>
      </div>
    </div>
  </div>
</section>

  { /* Call-to-Action Section - Final push to sign up with animated decorative
elements */ }
  <section id="cta" className="ssp-landing__section reveal-on-scroll">
    <div className="ssp-landing__cta-panel" ref={registerCard}>
      <div className="ssp-landing__cta-object stack-books">
        <span />
        <span />
        <span className="star" />
      </div>
      <div className="ssp-landing__cta-copy">
        <h2>Ready to Transform Your Study Journey?</h2>
        <p>Start with a cleaner study system built for actual academic
pressure.</p>
        <Link to="/register" className="ssp-landing__primary-btn"
ref={registerButton}>Get Started Free</Link>
      </div>
      <div className="ssp-landing__cta-object backpack">
        <span className="bag-body" />
        <span className="bag-pocket" />
        <span className="bag-handle" />
      </div>
    </div>
  </section>
</main>
</div>
);
}

```

File: backend/ server.js

```
// Load environment variables from .env file
require('dotenv').config();

// Import Express app
const app = require('./app');

// Import database connection function
const connectDB = require('./config/db');

// Set PORT from environment or default to 5000
const PORT = process.env.PORT || 5000;

// Connect to database first
connectDB().then(() => {

  // Start server after DB connection is successful
  const server = app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
  });

  // Graceful shutdown function (for SIGTERM / SIGINT)
  const shutdown = (signal) => {
    console.log(`${signal} received. Closing HTTP server.`);

    // Close server properly before exiting
    server.close(() => {
      console.log('HTTP server closed.');
```

```
      process.exit(0); // Exit process successfully
    });
  };

  // Handle termination signal (like from Docker, hosting platforms)
  process.on('SIGTERM', () => shutdown('SIGTERM'));

  // Handle interrupt signal (Ctrl + C)
  process.on('SIGINT', () => shutdown('SIGINT'));
});

// Handle unhandled promise rejections (async errors not caught)
process.on('unhandledRejection', (error) => {
  console.error(`Unhandled rejection: ${error.message}`);
  process.exit(1); // Exit with failure
});

// Handle uncaught exceptions (sync errors not caught)
```

```
process.on('uncaughtException', (error) => {
  console.error(`Uncaught exception: ${error.message}`);
  process.exit(1); // Exit with failure
});
```

File: backend/ controllers/ analyticsController.js

```
// Import Task model (stores user tasks)
const Task = require('../models/Task');

// Import MentalLoad model (stores fatigue, stress, etc.)
const MentalLoad = require('../models/MentalLoad');

// Import StudySchedule model (stores study hours data)
const StudySchedule = require('../models/StudySchedule');

// Import asyncHandler to handle async errors automatically
const asyncHandler = require('../utils/asyncHandler');

// Import utility functions for date manipulation
const { getEndOfDay, getStartOfDay } = require('../utils/date');

// Helper function to build date filter based on query params
const buildDateFilter = (userId, query) => {
  const filter = { userId }; // Base filter for user

  // If startDate or endDate is provided in query
  if (query.startDate || query.endDate) {
    filter.date = {};

    // Set start of date range
    if (query.startDate) filter.date.$gte = getStartOfDay(query.startDate);

    // Set end of date range
    if (query.endDate) filter.date.$lte = getEndOfDay(query.endDate);
  }

  return filter; // Return constructed filter object
};

// Controller to get study time analytics
const getStudyTimeAnalytics = asyncHandler(async (req, res) => {

  // Fetch study schedules within date range
  const schedules = await StudySchedule.find(buildDateFilter(req.user._id, req.query));

  // Fetch completed tasks of the user
```

```

const tasks = await Task.find({ createdBy: req.user._id, status: 'completed' });

// Calculate total study hours
const totalStudyHours = schedules.reduce((sum, schedule) => sum +
schedule.totalStudyHours, 0);

// Count completed tasks
const completedTasks = tasks.length;

// Send response with analytics data
res.json({
  totalStudyHours: Number(totalStudyHours.toFixed(2)), // Rounded to 2 decimals
  completedTasks,
  averageDailyStudyHours: schedules.length
    ? Number((totalStudyHours / schedules.length).toFixed(2)) // Average per day
    : 0
});
});

// Controller to get fatigue & mental load analytics
const getFatiguePatternAnalytics = asyncHandler(async (req, res) => {

  // Fetch mental load entries sorted by date
  const entries = await MentalLoad.find(buildDateFilter(req.user._id,
req.query)).sort({ date: 1 });

  // Create trend data (daily entries)
  const trend = entries.map((entry) => ({
    date: entry.date,
    fatigueLevel: entry.fatigueLevel,
    stressLevel: entry.stressLevel,
    motivationLevel: entry.motivationLevel,
    sleepHours: entry.sleepHours
  }));

  // Calculate averages if entries exist
  const averages = entries.length
    ? {
      fatigueLevel: Number(
        (entries.reduce((sum, item) => sum + item.fatigueLevel, 0) /
entries.length).toFixed(2)
      ),
      stressLevel: Number(
        (entries.reduce((sum, item) => sum + item.stressLevel, 0) /
entries.length).toFixed(2)
      ),
      motivationLevel: Number(
        (entries.reduce((sum, item) => sum + item.motivationLevel, 0) /

```

```

entries.length).toFixed(2)
    ),
    sleepHours: Number(
      (entries.reduce((sum, item) => sum + item.sleepHours, 0) /
entries.length).toFixed(2)
    )
  }
  : { fatigueLevel: 0, stressLevel: 0, motivationLevel: 0, sleepHours: 0 }; //
Default if no data

  // Send response
  res.json({ averages, trend });
});

// Controller to calculate overall productivity score
const getProductivityScore = asyncHandler(async (req, res) => {

  // Run all DB queries in parallel for performance
  const [completedTasks, pendingTasks, schedules, mentalLoads] = await Promise.all([

    // Count completed tasks
    Task.countDocuments({ createdBy: req.user._id, status: 'completed' }),

    // Count pending tasks
    Task.countDocuments({ createdBy: req.user._id, status: 'pending' }),

    // Fetch study schedules
    StudySchedule.find(buildDateFilter(req.user._id, req.query)),

    // Fetch mental load entries
    MentalLoad.find(buildDateFilter(req.user._id, req.query))
  ]);

  // Calculate task completion rate
  const taskCompletionRate =
    completedTasks + pendingTasks ? completedTasks / (completedTasks + pendingTasks) :
0;

  // Calculate average study hours
  const averageStudyHours = schedules.length
    ? schedules.reduce((sum, schedule) => sum + schedule.totalStudyHours, 0) /
schedules.length
    : 0;

  // Calculate average fatigue (default 5 if no data)
  const averageFatigue = mentalLoads.length
    ? mentalLoads.reduce((sum, entry) => sum + entry.fatigueLevel, 0) /
mentalLoads.length

```

```

: 5;

// Calculate productivity score (bounded between 0 and 100)
const productivityScore = Math.max(
  0,
  Math.min(
    100,
    Math.round(
      taskCompletionRate * 55 + // Weight for task completion
      averageStudyHours * 8 +   // Weight for study hours
      (10 - averageFatigue) * 5 // Lower fatigue = higher score
    )
  )
);

// Send response with score and summary
res.json({
  productivityScore,
  summary: {
    completedTasks,
    pendingTasks,
    averageStudyHours: Number(averageStudyHours.toFixed(2)),
    averageFatigue: Number(averageFatigue.toFixed(2))
  }
});

// Export all controllers
module.exports = {
  getStudyTimeAnalytics,
  getFatiguePatternAnalytics,
  getProductivityScore
};

```


File: backend/ controllers/ mentalLoadController.js

```
// Import MentalLoad model (stores fatigue, stress, motivation data)
const MentalLoad = require('../models/MentalLoad');

// Import Task model (used for calculating workload factors)
const Task = require('../models/Task');

// Import StudySchedule model (used to track overloaded days)
const StudySchedule = require('../models/StudySchedule');

// Import asyncHandler to handle async errors automatically
const asyncHandler = require('../utils/asyncHandler');

// Import date utility functions
const { getEndOfDay, getStartOfDay } = require('../utils/date');

// Import function to estimate stress & motivation
const { estimateMentalState } = require('../utils/estimateMentalState');

// Controller to create or update (upsert) mental load entry
const upsertMentalLoad = asyncHandler(async (req, res) => {

  // Normalize input date to start of the day
  const targetDate = getStartOfDay(req.body.date);

  // Create a range of last 7 days (including target date)
  const rangeStart = new Date(targetDate);
  rangeStart.setDate(rangeStart.getDate() - 6);

  // Fetch multiple task-related metrics in parallel
  const [
    pendingTasks,
    overdueTasks,
    heavyPendingTasks,
    completedRecently,
    overloadedDays
  ] = await Promise.all([

    // Count all pending tasks
    Task.countDocuments({
      createdBy: req.user._id,
      status: 'pending'
    }),
```

```

// Count overdue tasks (deadline passed)
Task.countDocuments({
  createdBy: req.user._id,
  status: 'pending',
  deadline: { $lt: targetDate }
}),

// Count heavy tasks (high difficulty or long duration)
Task.countDocuments({
  createdBy: req.user._id,
  status: 'pending',
  $or: [
    { difficultyLevel: { $gte: 4 } },
    { estimatedDuration: { $gte: 2 } }
  ]
}),

// Count tasks completed in last 7 days
Task.countDocuments({
  createdBy: req.user._id,
  status: 'completed',
  updatedAt: { $gte: rangeStart, $lte: getEndOfDay(targetDate) }
}),

// Count overloaded days in last 7 days
StudySchedule.countDocuments({
  userId: req.user._id,
  date: { $gte: rangeStart, $lte: getEndOfDay(targetDate) },
  workloadLevel: 'overloaded'
})
]);

// Estimate stress & motivation based on behavior + input
const estimatedState = estimateMentalState({
  fatigueLevel: Number(req.body.fatigueLevel),
  sleepHours: Number(req.body.sleepHours),
  pendingTasks,
  overdueTasks,
  heavyPendingTasks,
  completedRecently,
  overloadedDays
});

// Create or update mental load entry in DB
const mentalLoad = await MentalLoad.findOneAndUpdate(
  { userId: req.user._id, date: targetDate }, // Match condition

```

```

    {
      userId: req.user._id,
      date: targetDate,
      fatigueLevel: Number(req.body.fatigueLevel), // User input
      sleepHours: Number(req.body.sleepHours),    // User input
      stressLevel: estimatedState.stressLevel,     // AI estimated
      motivationLevel: estimatedState.motivationLevel // AI estimated
    },

    {
      new: true,                // Return updated document
      upsert: true,             // Create if not exists
      runValidators: true,      // Apply schema validation
      setDefaultsOnInsert: true // Apply defaults if inserting
    }
  );

  // Send response with saved data and context
  res.status(201).json({
    message: 'Mental load recorded successfully with behavior-based stress and
motivation estimates',
    mentalLoad,

    // Additional data used for estimation (debug / insight)
    estimationContext: {
      pendingTasks,
      overdueTasks,
      heavyPendingTasks,
      completedRecently,
      overloadedDays
    }
  });
});

```

```
// Controller to get all mental load entries (with optional date filter)
const getMentalLoads = asyncHandler(async (req, res) => {
```

```
  const filter = { userId: req.user._id }; // Base filter
```

```
  // Apply date range filter if provided
```

```
  if (req.query.startDate || req.query.endDate) {
    filter.date = {};
```

```
    if (req.query.startDate) {
      filter.date.$gte = getStartOfDay(req.query.startDate);
    }

```

```
    if (req.query.endDate) {
      filter.date.$lte = getEndOfDay(req.query.endDate);
    }
  }
}
```

```
// Fetch entries sorted by latest date first
```

```
const entries = await MentalLoad.find(filter).sort({ date: -1 });
```

```
// Send response
```

```
res.json({
  count: entries.length,
  mentalLoads: entries
});
});
```

```
// Controller to get mental load for a specific date
```

```
const getMentalLoadByDate = asyncHandler(async (req, res) => {
```

```
  // Normalize date from params
```

```
  const targetDate = getStartOfDay(req.params.date);
```

```
  // Find entry for that date
```

```
  const mentalLoad = await MentalLoad.findOne({
    userId: req.user._id,
    date: targetDate
  });
});
```

```
// If not found → throw error
if (!mentalLoad) {
  res.status(404);
  throw new Error('Mental load entry not found for the selected date');
}

// Send response
res.json({ mentalLoad });
});

// Export all controllers
module.exports = {
  upsertMentalLoad,
  getMentalLoads,
  getMentalLoadByDate
};
```